UET NOWSHERA | ELECTRICAL ENGINEERING

LAB REPORT 07

ONE-DIMENSIONAL ARRAYS IN C++

COURSE: Computer Programming

AUTHOR: Qazi Ehsanullah

ROLL NO: BF25NWELE0738

1. OBJECTIVE

- To understand what an array is and why it is used in C++.
- To learn how to declare and initialize one-dimensional arrays.
- To access, read, and modify individual array elements using index notation.
- To copy elements from one array to another using a loop.
- To work with character arrays and understand how individual characters are stored and accessed.

2. THEORETICAL BACKGROUND

What is an Array? An array is a data structure in C++ that stores a fixed-size sequential collection of elements all of the same type. Rather than declaring a separate variable for each value, an array groups multiple values under a single name and allows each value to

be accessed by its numeric index. All array elements occupy contiguous memory locations, with the lowest address corresponding to the first element.

Declaration & Initialization. An array is declared by specifying the data type, a name, and the number of elements in square brackets, such as int age[4];. An array can be initialized at the time of declaration by providing a comma-separated list of values enclosed in curly braces. Alternatively, individual elements can be assigned values after declaration using index notation.

Accessing & Copying Array Elements. Individual elements are accessed using the array name followed by the index in square brackets. Array indices in C++ begin at 0, so the first element is at index 0 and the last element of an array of size n is at index n-1. C++ does not provide a single statement to copy an entire array. Each element must be copied individually using a loop, iterating through every index to assign each source element to the corresponding position in the destination array.

Character Arrays. A character array stores a sequence of characters, and in C++, a word can be represented as a character array where each element holds one character. Individual characters can be accessed and printed using the same index notation as any other array.

3. SOFTWARE USED

- IDE: DEV-C++ IDE
- Compiler: C++ Compiler (bundled with DEV-C++)

4. EXPERIMENTAL TASKS

Task 1: Array Declaration and Initialization

An integer array named numbers of size 5 was declared and initialized with the values 10, 20, 30, 40, and 50. A for loop was then used to iterate through the array and display each element along with its index. The implementation is as follows:

```cpp
#include "str"><iostream>
using namespace std;

int main() {
    // declare and initialise a 5-element integer array
    int numbers[5] = {10, 20, 30, 40, 50};

    // display each element using its index
    for (int i = 0; i < 5; i++) {
        cout << "numbers[" << i << "] = " << numbers[i] << endl;
    }
    return 0;
}
```

Task 2: Accessing Character Array Elements

A character array named message was declared and initialized with the word 'POWER'. A for loop was used to access each character of the array one by one and print it on a separate line, demonstrating individual element access in a character array. The implementation is as follows:

```cpp
#include "str"><iostream>
using namespace std;

int main() {
    char message[] = "POWER"; // character array storing a word
    int len = 5; // number of characters to display

    // print each character on its own line
    for (int i = 0; i < len; i++) {
        cout << message[i] << endl;
    }
    return 0;
}
```

Task 3: Copying Arrays

Two integer arrays, source and destination, each of size 5, were declared. The source array was initialized with five values. A for loop was used to copy each element of source into the corresponding position in destination. Both arrays were then displayed to verify the copy operation. The implementation is as follows:

```cpp
#include "str"><iostream>
using namespace std;

int main() {
    int source[5] = {5, 10, 15, 20, 25}; // values to copy
    int destination[5]; // receives copied values

    // copy each element one by one
    for (int i = 0; i < 5; i++) {
        destination[i] = source[i];
    }

    cout << "Source: ";
    for (int i = 0; i < 5; i++) cout << source[i] << " ";
    cout << endl;

    cout << "Destination: ";
    for (int i = 0; i < 5; i++) cout << destination[i] << " ";
    cout << endl;

    return 0;
}
```

Task 4: Array Sum

This program prompts the user to enter 5 integers one at a time. Each value is stored in an integer array. After all values have been entered, a second for loop computes the total sum of all elements and displays the result. The implementation is as follows:

```cpp
#include "str"><iostream>
using namespace std;

int main() {
    int nums[5];
    int sum = 0;

    // read 5 integers into the array from the user
    for (int i = 0; i < 5; i++) {
        cout << "Enter number " << i+1 << ": ";
        cin >> nums[i];
    }

    // accumulate the sum of all elements
    for (int i = 0; i < 5; i++) {
        sum += nums[i];
    }

    cout << "Sum = " << sum << endl;
    return 0;
}
```

5. CONCLUSION

This lab introduced one-dimensional arrays in C++, covering declaration, initialization, element access, array copying, and character arrays. Arrays were shown to be an efficient way to manage collections of related values under a single variable name. Loop structures were used throughout to traverse and process array elements, reinforcing the close relationship between arrays and iteration. These fundamentals form the foundation for working with more advanced data structures encountered in later stages of the course.